Методичні вказівки
до виконання лабораторної роботи №1
на тему «Вступ до Apache Spark та огляд парадигми Map

Reduce» з курсу

“Обробка надвеликих масивів даних”

Київ – 2023

Мета роботи

Ознайомитися з основними поняттями та можливостями Apache Spark та алгоритму MapReduce, отримати базові навички налаштування середовища PySpark для обробки великих даних.

План роботи

1. Отримати розуміння інструментів, необхідних для виконання даного завдання
2. Налаштувати середовище
3. Запустити і зрозуміти логіку роботи програми “word count”
4. Реалізувати та зрозуміти алгоритму “inverted index”
5. Теоретично розібрати бібліотеку Apache Spark, файлову систему HDFS
6. Підготувати протокол з результатами виконання роботи, який має містити наступні пункти:
    6.1. Кроки налаштування середовища
    6.2. Запуск та пояснення роботи програми «word count»
    6.3. Запуск та пояснення роботи програми «inverted index» 6.4. Відповідь на теоретичні запитання

Налаштування середовища

1. Встановити Java Development Kit (JDK) на комп'ютері. PySpark працює на Java, тому без JDK він не буде працювати.
2. Встановити Apache Spark на комп'ютері. Можна завантажити останню версію з офіційного сайту Apache Spark https://spark.apache.org/downloads.html .
3. Встановити Python на комп'ютері. PySpark підтримує Python версії 2.7 та 3.4 і вище.
4. Встановити PySpark на комп'ютері. Це можна зробити за допомогою pip, якщо він вже встановлений на комп'ютері. Команда для встановлення PySpark: pip install pyspark.
5. Налаштувати змінні середовища. Для цього потрібно додати шлях до папки з Apache Spark та PySpark до змінної середовища PATH.

Наприклад, якщо Apache Spark встановлено в папку C:\spark, а PySpark встановлено за допомогою pip, то шлях до PySpark буде C:\Users\username\AppData\Local\Programs\Python\Python36-32\Lib\site-packages\pyspark, де username - це ім'я користувача на комп'ютері.

6. Запустити PySpark. Це можна зробити за допомогою командного рядка. Наприклад, для запуску PySpark в режимі локального виконання потрібно ввести команду pyspark в командному рядку.

Word count вікористовуючи парадигму Map Reduce

Map Reduce може бути використаний для багатьох задач, наприклад для обробки файлів які можна розбити на рядки (наприклад, текстові файли) у задачі для підрахунку слів у файлах. Така задача є свого роду «Hello, World!» у світі великих даних

Приклад:
Наприклад у нас є 2 файла "A.txt" і "B.txt" з таким змістом: *A.txt*

This is the A file, it has words in it
B.txt

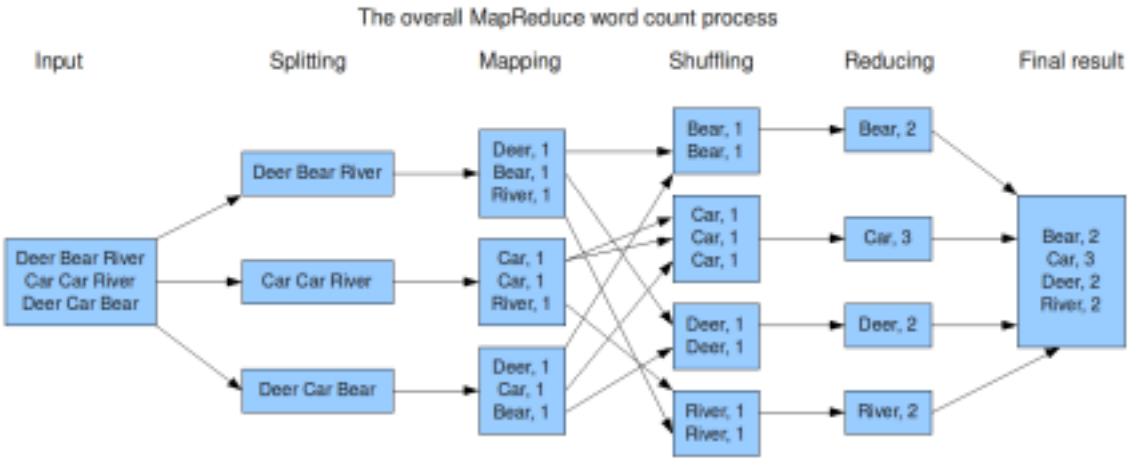
Welcome to the B file, it has words too
The algorithm we use to count the words must fit in MapReduce. We will implement the
following pseudo-code algorithm:
Mapper: takes as input a line from a document
foreach word w in line:
emit (word, 1)
Reducer: takes as input a key (word) and a set of values (all of which will be "1") sum = 0
foreach v in values:
sum = sum + v
emit (word, sum)

Маперам передаються всі рядки з кожного файлу, по одному за раз. Ми

розбиваємо їх на слова і виділяємо (слово, 1) пари - вказуючи, що в той момент ми побачили це слово один раз. Потім ред'юсер збирає всі "1", і конкатенує файли для того шоб порахувати слова.
Відповідно на виході з маперів ми отримаємо:
Мапер для файлу A.txt:
< This, 1 >
< is, 1 >
< the, 1 >
< A, 1 >
< file, 1 >
< it, 1 >
< has, 1 >
< words, 1 >
< in, 1 >
< it, 1 >
Мапер для файлу B.txt:
< Welcome, 1 >
< to, 1 >
< the, 1 >
< B, 1 >
< file, 1 >
< it, 1 >
< has, 1 >
< words, 1 >
< too, 1 >
Кожне слово має свій власний процес reduce. Наприклад, для слова "it" він буде виглядати так:
key="it"; values=iterator[1, 1, 1]
І отримаємо на виході:
< it, 3 >

Порядок виконання (Word Count):
Використовуючи вхідний текст, запустіть Word Count за допомогою Hadoop, щоб отримати зведену кількість слів у кожному з декількох документів.
Основна концепція Word Count описана вище, а також зображена на

малюнку нижче. З детельним туторіалом можна ознайомитися на офіційній сторінці: *https://hadoop.apache.org/docs/r2.7.1/hadoop-mapreduce client/hadoop-mapreduce-client core/MapReduceTutorial.html#Example:_WordCount_v2.0*



(https://cs.calvin.edu/courses/cs/374/exercises/12/lab/)

Inverted Index

Інвертований індекс — структура даних, в якій для кожного слова колекції документів у відповідному списку перераховані всі документи в колекції — в яких воно зустрілося. Інвертований індекс використовується для пошуку за текстами.

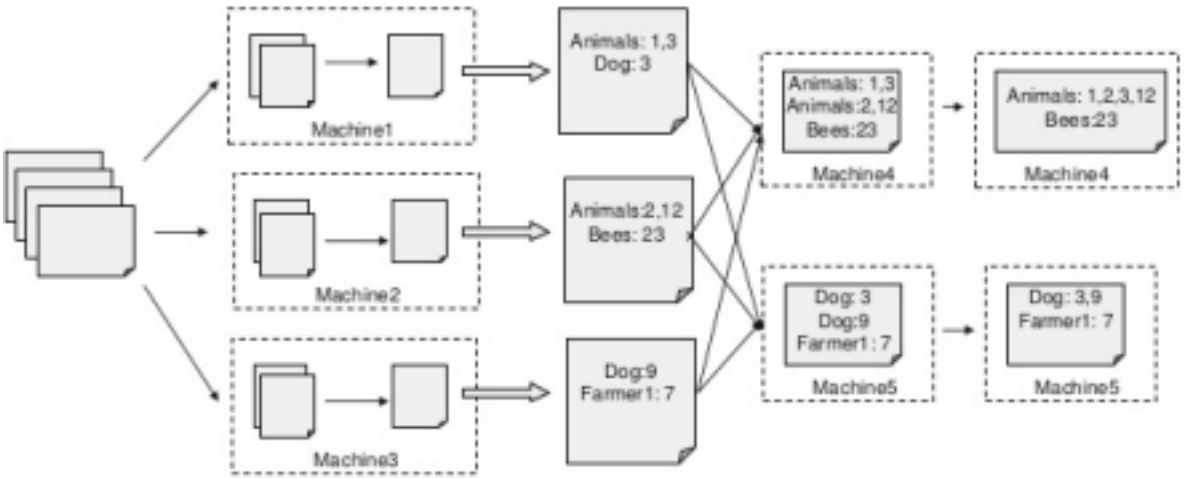
Є два варіанти інвертованого індексу:

- індекс, який містить лише список документів для кожного слова,
- індекс, додатково включає позицію слова в кожному документу

Вам необхідно створити інвертований індекс для файлів які ви використовували в word count.
З урахуванням вхідного тексту індексатор зсуву використовує Hadoop для створення індексу всіх слів у тексті. Для кожного слова індекс має список усіх місць, де з'являється це слово, і, можливо, текстовий витяг кожного рядка, де слово з'являється. Запустіть індексатор ліній на творах Шекспіра:
lucrece.txt, offset 38624: To cipher what is writ in learned books,

orlando.txt, offset 66001: ORLANDO Which I take to be either a fool or a cipher. would produce the following output for the word "cipher:"
cipher lucrece.txt@38624,orlando.txt@66001,...

Порядок виконання (Inverted Index):



1. Створіть новий проект
2. Створіть клас мапера який буде приймати такі дані:

    1. LongWritable key - побайтовий зсув поточної строчки в файлі
    2. Text value - поточна строчка
    3. OutputCollector - output - .collect метод для виведення <key, value> пари
    4. Reporter reporter

Мапер має на виході отримувати пари <"word", "filename@offset">

3. Створіть клас reduce який буде приймати дані від мапера 4. Створіть драйвер - програму що буде імплементувати main() функцію (за тим же принципом що і в випадку з word count)

Теоретичні питання

1. Що таке великі дані (Big Data) та як вони відрізняються від звичайних даних? Які проблеми виникають при обробці великих

даних? 2. Як працює алгоритм MapReduce? Які етапи включає цей алгоритм та як вони пов'язані між собою?

3. Що таке Apache Spark та як він пов'язаний з алгоритмом MapReduce? Які компоненти входять до складу Apache Spark та як вони взаємодіють між собою?

4. Як працює PySpark? Які основні поняття та технології використовуються в PySpark для обробки великих даних? 5. Які є методи обробки даних в PySpark? Які операції можна виконувати з даними в PySpark та як вони впливають на швидкість обробки даних?